Chapter 4

The MHE–NMPC Framework

Chapter 3 established *what* is estimated and why. This chapter states *how*: the numerical form of the two optimal control problems, the interface that carries the estimate from one to the other, and the logic that decides when the controller may believe that its payload has gone.

The estimator receives no attach or release notification. The controller's model parameters follow the estimate continuously, and a release is *accepted* — declared complete, with auxiliary states reset — only on persistent estimator evidence. Section 4.4.1 separates what that acceptance does from what the continuous adaptation does, and Section 4.4.6 states what a mass-only lifecycle cannot do.

4.1 Closed-Loop Architecture

Figure 4.1 shows the information flow. Three layers are involved: the physics and flight stack (Gazebo, PX4 software-in-the-loop, and a magnetic detachable-joint gripper that produces genuine mass and inertia changes); the inner rate-control loop on the autopilot; and a ROS outer loop containing the estimator and the predictive controller.

Two properties of this diagram carry the argument of the dissertation.

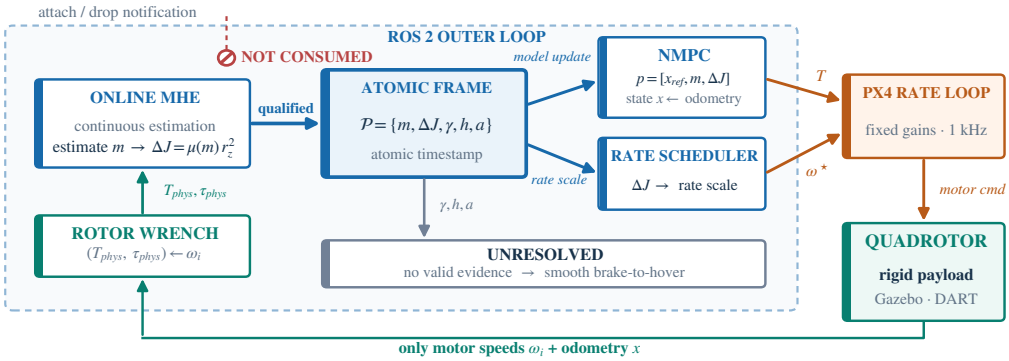


Figure 4.1: System architecture and information flow. The complete pipeline consumes only rotor speeds and odometry. One atomic, health-gated frame carrying the composite mass and the inertia increment derived from it drives both the NMPC model and the continuous body-rate command scheduling. No attach or release notification enters this path.

First, **the only inputs are odometry and rotor speeds.** The estimator does not read the controller's intended thrust or torque (Section 3.5), does not read the gripper's attach or release record, and does not read the simulator's joint state. The payload truth available in simulation is used exclusively for offline scoring.

Second, **there is exactly one interface.** The estimator publishes a single atomic frame (Section 4.3) containing the mass, the derived inertia, a no-payload evidence score and a health flag. Both consumers — the NMPC model parameters and the inner-loop command scaling — read that one frame. An earlier design in which mass and inertia travelled on separate topics, each with its own latency and validity, made it possible for the controller to hold a mutually inconsistent parameter set; the atomic frame makes that class of fault unrepresentable.

The two optimisers run at different rates and do not share a solve clock. The estimator solves at 10 Hz over a 2 s window; the controller solves at 20 Hz over a 1 s preview; setpoints are published at 50 Hz by interpolating along the most recent predicted trajectory.

4.2 Moving Horizon Estimator

4.2.1 The optimisation

With the augmented state (3.21) of Chapter 3, the estimator solves, at each 10 Hz tick,

$$\begin{aligned} \min_{x_{0:N},\, w_{0:N-1}} \quad & \left\|x_0 - \bar{x}_0\right\|_{W_0}^2 + \sum_{j=0}^{N-1} \beta_j \left(\left\|y_j - C x_j\right\|_{W_y}^2 + \left\|w_j\right\|_{W_w}^2 \right) \\ \text{s.t.} \quad & x_{j+1} = F_{\mathrm{M}}\left(x_j, u_j^{\mathrm{phys}}\right) + G w_j, \\ & \underline{m} \le m_T \le \overline{m}, \end{aligned} \tag{4.1}$$

where y_j is the odometry measurement, C selects the thirteen vehicle states from the fourteen-dimensional augmented state, F_{M} is one explicit fourth-order Runge–Kutta step of the dynamics of Chapter 3 driven by the reconstructed wrench u^{phys} of (3.24), and

$$G = \begin{bmatrix} I_{13} \\ 0_{1\times 13} \end{bmatrix} \tag{4.2}$$

confines the process noise to the physical states, leaving m_T noise-free within the window.

The horizon is $N = 20$ stages at $\Delta t = 0.1\,\mathrm{s}$. The choice is a compromise: a longer window averages more measurement noise, but the constraint that makes the mass identifiable — that it is *constant* over the window — becomes less defensible the longer the arc of trajectory the window spans. In development flights in an earlier configuration, on a $4\,\mathrm{m\,s^{-1}}$ figure-eight a ten-stage window recovered the mass $3.7\times$ more accurately than a twenty-stage one, while a forty-stage window sat on the lower bound for the entire flight. The horizon was not varied in the evaluated configuration.

4.2.2 Weights

All three weight matrices follow Bryson's rule, with each diagonal entry set by a representative residual magnitude. This keeps the residual blocks on comparable numerical scales.

The measurement weight uses representative state-estimation standard deviations,

$$W_y = \mathrm{diag}\Big(\sigma_p^{-2} I_3,\ \sigma_v^{-2} I_3,\ \sigma_q^{-2} I_4,\ \sigma_\omega^{-2} I_3\Big), \tag{4.3}$$

with $\sigma_p = 0.02\,\mathrm{m}$, $\sigma_v = 0.05\,\mathrm{m\,s^{-1}}$, $\sigma_q = 0.01$ (about 1.1°) and $\sigma_\omega = 0.02\,\mathrm{rad\,s^{-1}}$.

The process-noise weight is deliberately *tighter* than the measurement weight,

$$W_w = \mathrm{diag}\big(10^4 I_6,\ 10^5 I_4,\ 10^3 I_3\big). \tag{4.4}$$

The tight translational block forces persistent disagreement between measured motion and reconstructed thrust to be explained by the mass rather than by unstructured process noise. The angular block is one decade looser because the rotational equation is numerically more delicate and carries the modelling errors identified in Section 3.4.2.

The arrival cost anchors the start of the window,

$$W_0 = \mathrm{diag}\big(10^3 I_6,\ 10^4 I_4,\ 10^2 I_3,\ \sigma_{m,0}^{-2}\big), \qquad \sigma_{m,0} = 3.16\,\mathrm{kg}, \tag{4.5}$$

with $\bar{x}_0$ taken from the previous window shifted by one stage. The mass anchor is deliberately four to five orders of magnitude softer than the physical-state entries: it carries negligible prior information but remains non-zero so that a poorly excited window retains positive curvature in the mass direction. Setting it to zero produced 472 solver failures in an earlier configuration.

The composite mass is constrained to $[0.95 m_B,\ 5.0\,\mathrm{kg}] = [1.961\,\mathrm{kg}, 5.0\,\mathrm{kg}]$: the

vehicle cannot weigh less than its airframe, while the 5 % margin leaves room for calibration error. The MHE imposes this interval as a hard constraint.

Prior-free ablation. A switch removes the remaining airframe-mass information by lowering the bound to 0.2 kg and setting $\sigma_{m,0} = 1000\,\mathrm{kg}$. It is an ablation, not the evaluated default; the measurement-derived seed below is prior-free in either configuration.

4.2.3 Initialisation

The estimator requires an initial iterate for the mass channel. Supplying the calibrated airframe mass would be a prior about the answer; the implementation instead constructs the seed from measurements.

The simplest form is the hover approximation obtained by setting $a_z = 0$ and $\phi = \theta = 0$ in the vertical force balance (3.8),

$$m^{(0)} = \mathrm{clip}\left(\frac{T_{\mathrm{phys}}}{g},\, \underline{m},\, \overline{m}\right), \tag{4.6}$$

which is prior-free because T_{phys} comes from (3.24) and involves no model state. It is valid only near hover: in development flights on a $4\,\mathrm{m\,s^{-1}}$ figure-eight the instantaneous ratio T_{phys}/g swung from $-42\,\%$ to $30\,\%$ of the true mass, because the collective thrust also supplies centripetal acceleration and is tilted away from the vertical.

The deployed seed therefore uses the whole window. Summing (3.8) over the N stages,

$$m_T \sum_{i=0}^{N-1} \left(g + a_{z,i}\right) \;=\; \sum_{i=0}^{N-1} T_i \cos\phi_i \cos\theta_i, \tag{4.7}$$

and eliminating the accelerations by the telescoping identity

$$\sum_{i=0}^{N-1} a_{z,i} = \sum_{i=0}^{N-1} \frac{v_{z,i+1} - v_{z,i}}{\Delta t} = \frac{v_{z,N} - v_{z,0}}{\Delta t}, \tag{4.8}$$

gives a seed that requires no numerical differentiation of the velocity signal:

$$m^{(0)} = \frac{\sum_{i=0}^{N-1} T_i \cos\phi_i \cos\theta_i}{Ng + \dfrac{v_{z,N} - v_{z,0}}{\Delta t}}. \tag{4.9}$$

Only the two velocity endpoints of the window enter, so the seed is immune to the sample-to-sample noise a finite-difference acceleration would amplify; the tilt factor restores the vertical component that (4.6) discards; and every input is either T_{phys} or odometry, so the seed is decoupled from the estimator's own state. The implementation falls back to (4.6), and then to a neutral box value, when the window is not yet full or the denominator of (4.9) degenerates.

The seed enters the solver in two places: the initial iterate of the sequential quadratic programme, and the mean of the arrival cost. The former affects only the number of iterations, never the optimum. The latter affects the optimum in proportion to its weight, which by (4.5) corresponds to a standard deviation of several kilograms — far wider than the range of masses the vehicle can carry. The seed therefore influences where the optimiser starts, not where it converges. The same expression is used on re-initialisation after repeated solver failure (Section 4.2.5).

4.2.4 Stage weighting: what was tried and what is deployed

The constant-payload model is invalid when a grasp or release lies inside the window, so three alternatives to uniform weighting were implemented. Their role and disposition are summarised in Table 4.1; the experimental negative result belongs to Section 5.7.4.

Table 4.1: Stage-weight alternatives considered for the MHE.

Treatment	Status	Reason
Uniform, $\beta_j \equiv 1$	evaluated default	no event logic; accepts a transient of roughly one window
Event-triggered de-weighting	retired	collective-thrust steps also occur at trajectory changes and cannot identify a payload event
Exponential forgetting, $\beta_j = \lambda^{N-1-j}$	available, off ($\lambda = 1$)	shortens effective memory smoothly but did not justify another tuning dimension
Mass random walk	available, off	permits within-window change but degraded Hessian conditioning

The evaluated design therefore tolerates the change transient rather than attempting to detect it; the lifecycle below is designed around that delay.

4.2.5 Re-anchoring after repeated solver failure

After five consecutive failed solves the window is discarded and rebuilt: the physical states are reset to the current measurement and the mass is re-seeded from (4.9). Re-seeding from a measurement rather than from the last accepted estimate matters because a run of failures is itself evidence that the last estimate may be corrupt; the window balance is the only mass value available at that instant that does not depend on the estimator's own history.

4.3 The Continuous Payload Interface

4.3.1 One atomic frame

After each solve the estimator publishes

$$\mathscr{P} = \Big\{ m_T,\ \Delta J_{xx}, \Delta J_{yy}, \Delta J_{zz},\ J_{xx}, J_{yy}, J_{zz},\ \gamma,\ h,\ a \Big\}, \tag{4.10}$$

in which the inertia entries are computed from m_T by (3.18), with $\Delta J_{zz} = 0$ for a centred point mass; $\gamma \in [0,1]$ is the no-payload evidence score of Section 4.4.3, h is a health bit and a is the age of the most recent successful solution. The message layout additionally retains four horizontal-offset fields for compatibility with the generated solver; they are not consumed by the evidence score, by the lifecycle logic, or by the controller, which holds $\boldsymbol{c}_{xy} = \mathbf{0}$.

Health is a conjunction: the latest solve must have succeeded, and the odometry, rotor-speed and known-input streams must all be fresher than 0.30 s. The consumer applies one further test of its own, on the receive age of the frame itself (0.35 s), because a frame can be healthy at the moment it is produced and stale by the time it is used.

The asymmetry in how an unhealthy frame is treated is deliberate and is the safety property of the interface: *an unhealthy or stale frame freezes the last accepted model and resets the evidence-score persistence timer*. A degraded estimate can therefore delay a release confirmation, but it can never cause one.

4.3.2 Filtering and slew limiting

A valid frame is not consumed directly. Each channel passes a first-order low-pass filter and a separate rate limiter before it reaches the solver:

$$\alpha = 1 - e^{-\Delta t/\tau}, \qquad \hat{\theta} \leftarrow \hat{\theta} + \alpha\big(\theta^{\mathrm{tgt}} - \hat{\theta}\big), \qquad \hat{\theta} \leftarrow \hat{\theta} + \mathrm{clip}\big(\cdot,\ \pm\dot{\theta}_{\max}\Delta t\big), \tag{4.11}$$

with $\tau = 0.20\,\mathrm{s}$ on the controller side (0.25 s on the estimator's own output stage) and rate limits $0.60\,\mathrm{kg\,s^{-1}}$ for the mass and $0.080\,\mathrm{kg\,m^2\,s^{-1}}$ for the inertia increment.

The reason for a limiter in addition to a filter is that the two protect against different things. The filter attenuates noise; the limiter bounds how fast the

model underneath a receding-horizon optimisation may change. A parameter step between consecutive solves invalidates the warm start, and an optimiser that must re-converge from scratch at 20 Hz is an optimiser that occasionally does not converge in time. The limiter converts any step — including a genuine one at grasp — into a ramp the solver can track.

4.4 The Grasp–Release Lifecycle

4.4.1 What release acceptance does, and why it is gated on evidence

Four things happen around a release, and the design keeps them apart.

1. *The dynamic parameters* — the mass, the inertia increment and the inner-loop scale — follow the estimate frame by frame through the continuous interface of Section 4.3, in every lifecycle state. When the payload leaves, they return to the bare-airframe values because the estimate does, not because any decision switches them.
2. *The release lifecycle state* — carry, release pending, release accepted, or UNRESOLVED — is owned by the controller.
3. *The residual specific force* of Section 4.5.3 is reset when a release is accepted with a release command outstanding.
4. *Task completion* — the delivery is declared done — coincides with acceptance.

Release acceptance is the transition of item 2; it triggers items 3 and 4 and never sets the mass or the inertia. Acceptance is the only lifecycle decision of the controller, and the rest of this section concerns when it may be taken.

The controller issues the gripper-release command itself, and it would be simple to accept the release at that instant. The command and the physical detachment can, however, differ: a gripper can be slow to open, can jam, or can release only part of a load. A controller that accepts at the command declares the delivery complete and discards the residual it learned against the loaded vehicle while the load may still be on the hook. Gating acceptance on estimator evidence — *estimator-confirmed detachment* — separates "the command was issued" from "the payload has left", at the cost of a confirmation delay while the estimator accumulates evidence.

Two arming policies are implemented on the estimator's evidence. The *always-on* policy evaluates the rule throughout carry; the *command-armed* policy starts its persistence count only after the release command. Chapter 5 shows why, on the mass evidence, the latter is preferred, and that neither detects a load lost without a command. A third rule, described in Section 4.4.5, replaces the mass evidence with an independent force-balance test and enables it either by the command or by a watchdog armed on airborne load; it is the only rule in this dissertation that addresses uncommanded loss.

4.4.2 Two state machines

The lifecycle is split between the two nodes, as shown in Figure 4.2. The estimator owns payload *presence*. The controller owns the *release lifecycle* and accepts a release only on *confirmation*. The separation exists so that no single estimator-side transition can, by itself, declare a delivery complete.

Presence is a hysteretic EMPTY↔LOADED machine. It enters LOADED when the estimated payload mass exceeds $0.09\,\mathrm{kg}$ for $2\,\mathrm{s}$, with a directional thrust residual as a fast path; it returns to EMPTY when the payload mass stays below $0.02\,\mathrm{kg}$ for $5\,\mathrm{s}$ *while the vehicle is quiet* ($\|\boldsymbol{\omega}\| \leq 0.15\,\mathrm{rad\,s^{-1}}$, $\|\boldsymbol{v}_{xy}\| \leq 0.20\,\mathrm{m\,s^{-1}}$). The quiet-flight condition is there because the mass is least reliable in manoeuvre

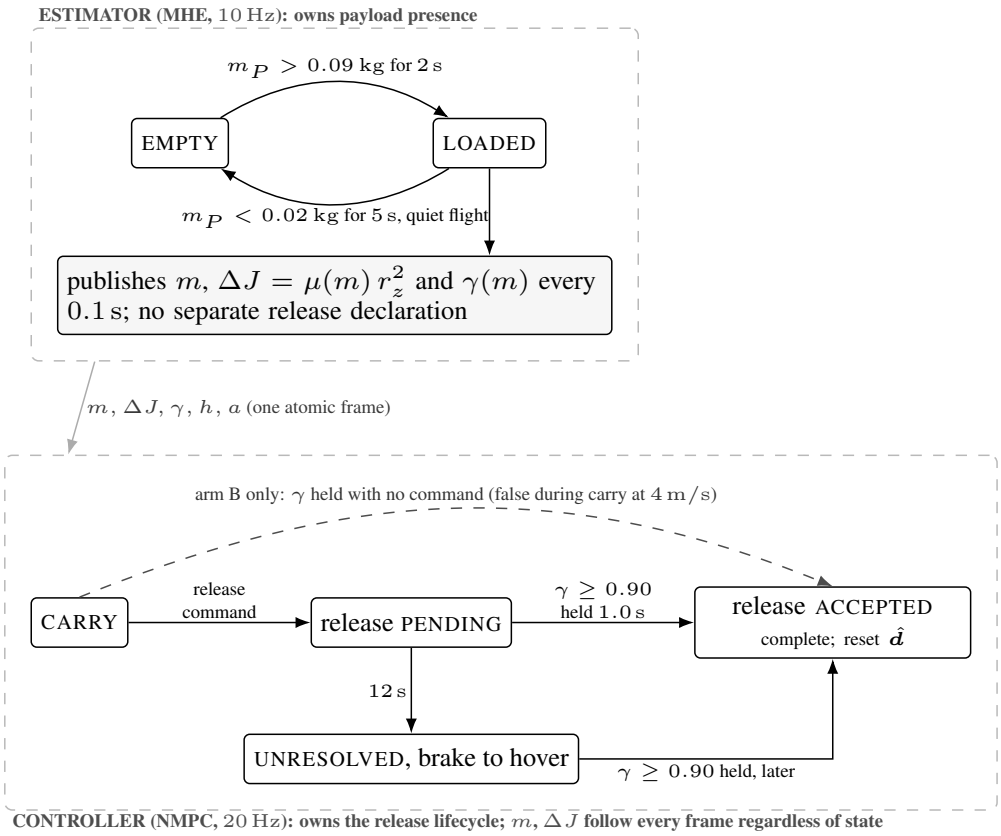


Figure 4.2: The payload lifecycle as two state machines. The estimator owns payload presence, driven by sustained payload-mass evidence with hysteresis. The controller owns the release lifecycle: the release command moves it from carry to release pending and starts the 12 s timeout, and a release is accepted on confirmation, $\gamma \geq 0.90$ held for 1.0 s. UNRESOLVED is a hold, not an acceptance: the vehicle brakes to hover and keeps waiting for evidence. The dashed path exists only under the always-on policy. In every state the mass and inertia increment follow the estimate frame by frame; no transition sets them.

(Section 4.4.6). The thresholds differ by a factor of four-and-a-half on purpose: the estimate is naturally low while it converges after a grasp, and a single threshold would toggle at the start of every lift. A separate latch records that the present flight reliably entered LOADED at some point.

4.4.3 No-payload evidence score

Apart from whether a release command has been issued, every lifecycle decision of the controller is a function of the published frame (4.10) alone. Let $\sigma(v; a, b)$

be the smooth indicator that is 1 for $|v| \le a$, 0 for $|v| \ge b$, and in between

$$\sigma(v;a,b) = 1 - \big(3\chi^2 - 2\chi^3\big), \qquad \chi = \frac{|v| - a}{b - a}, \tag{4.12}$$

which is the standard cubic smooth-step: continuous with continuous first derivative at both ends. Let $\bar{m}_{P,k}$ be the estimator's filtered payload mass and let $\gamma_k^\star$ be its instantaneous empty-payload score. The published score is filtered once more at the output stage:

$$\begin{aligned} \gamma_k^\star &= \sigma\big(\bar{m}_{P,k}; 0.015, 0.060\big), \\ \gamma_k &= \gamma_{k-1} + \alpha_\gamma\big(\gamma_k^\star - \gamma_{k-1}\big), \qquad \alpha_\gamma = 1 - e^{-\Delta t / 0.25\,\mathrm{s}}. \end{aligned} \tag{4.13}$$

Thus γ is a deterministic evidence score, not a posterior probability or a statistical confidence level. The score is 1 at or below 0.015 kg, 0 at or above 0.060 kg, and changes smoothly between them. The instantaneous score exceeds 0.90 below approximately 0.024 kg; the additional output filter determines when the published score crosses that threshold. The inertia increment is deliberately absent: in the mass-only model it is a deterministic function of the same mass estimate and therefore cannot provide independent release evidence.

4.4.4 Confirmation, UNRESOLVED, and braking to hover

The controller accepts a release through exactly one route:

$$\gamma \ge 0.90 \quad \text{held for} \quad 1.0\,\mathrm{s} \quad \text{on healthy, fresh frames.} \tag{4.14}$$

The persistence must be contiguous in valid estimator time: a stale or failed frame breaks the count rather than pausing it. There is no separate estimator declaration in the evaluated configuration, and the release command does not enter the evidence score (4.13); under the command-armed policy it only decides when the count of (4.14) may start.

If no confirmation arrives within 12 s of the release command, the controller enters UNRESOLVED. This state is what makes confirmation on the estimate deployable, and its semantics deserve emphasis: **a timeout is not a release**. Nothing is accepted, the residual is not reset, and the model continues to follow the estimate. What changes is the vehicle's exposure: it stops flying an aggressive trajectory it may no longer have the authority to fly, and it gives the estimator a quiet operating point in which the mass is best observed.

The braking manoeuvre is a finite-time reference rather than an instantaneous switch to hover, because at $4\,\mathrm{m\,s^{-1}}$ the latter would command a step of several metres per second. Let $\boldsymbol{p}_0, \boldsymbol{v}_0$ be the state at the declaration and $T_b = 3\,\mathrm{s}$. The stopping point is chosen as the displacement of a constant deceleration from $\boldsymbol{v}_0$ to rest,

$$\boldsymbol{p}_1 = \boldsymbol{p}_0 + \tfrac{1}{2} T_b \, \boldsymbol{v}_0, \tag{4.15}$$

and the reference between the two is the unique quintic satisfying $\boldsymbol{p}(0) = \boldsymbol{p}_0$, $\dot{\boldsymbol{p}}(0) = \boldsymbol{v}_0$, $\ddot{\boldsymbol{p}}(0) = \mathbf{0}$, $\boldsymbol{p}(T_b) = \boldsymbol{p}_1$, $\dot{\boldsymbol{p}}(T_b) = \mathbf{0}$, $\ddot{\boldsymbol{p}}(T_b) = \mathbf{0}$:

$$\begin{aligned} \boldsymbol{p}(t) = \boldsymbol{p}_0 + \boldsymbol{v}_0 t + a_3 t^3 + a_4 t^4 + a_5 t^5, \qquad & a_3 = \big(10\,\Delta\boldsymbol{p} - 6 T_b \boldsymbol{v}_0\big)/T_b^3, \\ & a_4 = \big(-15\,\Delta\boldsymbol{p} + 8 T_b \boldsymbol{v}_0\big)/T_b^4, \\ & a_5 = \big(6\,\Delta\boldsymbol{p} - 3 T_b \boldsymbol{v}_0\big)/T_b^5, \end{aligned} \tag{4.16}$$

with $\Delta\boldsymbol{p} = \boldsymbol{p}_1 - \boldsymbol{p}_0$. Zero acceleration at both ends means the manoeuvre begins and ends without a jump in commanded thrust, which is what keeps the transition from itself perturbing the estimator whose evidence is being awaited. Attitude is blended over the same interval by the smooth-step of (4.12).

4.4.5 Force-balance release evidence and the uncommanded-loss watchdog

The failure of the always-on rule (Section 4.4.6) lies in its evidence, not its arming: the mass estimate is the one quantity that degrades in manoeuvre. A second test of the same question needs evidence that does not pass through the attitude, the velocity history or the rotational model. The vertical force balance in the body frame provides it. The specific force measured by the inertial unit along the thrust axis, f_z, contains no gravity term, and with the rotor-drag term neglected it satisfies $f_z \approx T/m$. A mass can therefore be formed from two measured signals alone,

$$m_{\mathrm{acc},k} = \frac{\bar{T}_k}{\max\left(\bar{f}_{z,k},\, 1\,\mathrm{m\,s^{-2}}\right)}, \qquad T_k = \textstyle\sum_i k_f\, \omega_{i,k}^2, \tag{4.17}$$

where $\bar{(\cdot)}$ denotes a first-order low-pass with a time constant of 0.5 s, applied to the thrust reconstructed from rotor speeds and to f_z alike. The lower limit on $\bar{f}_z$ guards against division in free fall. No estimator state enters (4.17); in particular, it is unaffected by the manoeuvre-induced collapse of $\hat{m}$, and it remains available when the estimator's frames are unhealthy. Its weakness is the one the mass channel also has: a relative error in k_f scales m_{acc} one-for-one.

The evidence score has the form of (4.13) with the same pair of thresholds,

$$\gamma^{\star}_{\mathrm{acc},k} = \sigma\left((m_{\mathrm{acc},k} - m_B)^{+};\, 0.015,\, 0.060\right), \tag{4.18}$$

filtered at the output with a 0.25 s time constant. The score starts at zero, because no evidence is not evidence of an empty vehicle, and it is held rather than advanced while either signal is stale (older than 0.2 s); a stale frame also breaks the persistence count of (4.14), which is otherwise applied to γ_{acc} unchanged: $\gamma_{\mathrm{acc}} \geq 0.90$ held for 1.0 s. Under this rule the controller's lifecycle reads the inertial unit and the rotor speeds directly; the statement of Sec-

tion 4.4.3 that every lifecycle decision is a function of the published frame does not hold for it, by design.

What may consume the score is decided by *dual enabling*. The score is computed throughout the flight, but a confirmation counts only when one of two conditions holds:

1. a release command is pending — the command opens confirmation at once, as in the command-armed policy; or
2. the *uncommanded-loss watchdog* is armed. It arms after the lift has started, once fresh evidence of load, $\gamma_{\text{acc}} < 0.90 - 0.15 = 0.75$, has persisted for 2.0 s. Once armed it is not disarmed by a rising score — that rise is the event it exists to detect — and it is cleared only at the next pickup.

During grasp and ground support neither condition holds, so a parcel that is attached but still resting on the ground, for which the force balance correctly reads an empty vehicle, cannot produce an acceptance.

An acceptance with a command pending is a normal release, with the actions of Section 4.4.1. An acceptance with no command pending and the watchdog armed is declared an *uncommanded loss*. In addition to resetting the residual specific force, the controller then zeroes the payload geometry, ends the attach-time inertia floor and resets the inner-loop scale to the bare-airframe value, and suppresses any later planned release, since there is nothing left to release. It issues no gripper command, because the gripper's state is unknown and it may be the gripper that failed.

4.4.6 What a mass-only lifecycle cannot do

Because (4.13) is a test on the mass alone, the lifecycle inherits every failure of that estimate. In sustained manoeuvre a low bias can carry $\hat{m}_P$ below the 0.024 kg confirmation threshold while the payload is attached; no rule admitting only that estimate can distinguish the collapse from a real release.

The evaluated configuration holds the inertia floor until the release command, protecting inner-loop authority but not the lifecycle. Command-arming separately prevents carry-phase acceptance by not evaluating the release rule before a command. The cohort supports that arming policy (Section 5.5.4), but neither mitigation corrects the mass estimate itself. The force-balance rule of Section 4.4.5 leaves the estimate as it is and stops asking it the release question; its own limitation is calibration rather than manoeuvre (Section 5.6.4).

4.4.7 Threshold provenance

Table 4.2 lists every lifecycle threshold and where its value came from. None was selected against the evaluation data; all are engineering choices whose closed-loop sensitivity has not been swept, and they are reported as such.

Table 4.2: Lifecycle thresholds and their provenance.

Threshold	Value	Provenance
presence entry	$m_P > 0.09\,\mathrm{kg}$, 2 s	below the smallest payload tested; persistence suppresses the post-grasp convergence transient
presence exit	$m_P < 0.02\,\mathrm{kg}$, 5 s, quiet flight	slow backstop; hysteresis against the entry threshold
no-payload evidence score	(4.13); $\gamma \geq 0.90$ held 1.0 s	engineering choice
force-balance score	(4.17)–(4.18); low-pass 0.5 s/0.25 s; fresh within 0.2 s	thresholds shared with (4.13); time constants an engineering choice, checked by offline replay of the earlier cohort and frozen before the campaign of Section 5.6
loss-watchdog arming	$\gamma_{\mathrm{acc}} < 0.75$ for 2.0 s, after lift start	engineering choice; frozen before that campaign
UNRESOLVED timeout / brake	12 s / 3 s	engineering choice

4.5 Adaptive NMPC

4.5.1 The optimal control problem and its input baseline

Over a prediction horizon of N stages the controller solves

$$\begin{aligned} \min_{u_{0:N-1}} \quad & \sum_{k=0}^{N-1} \left(\left\| e_{\mathrm{track}}(x_k, x_{\mathrm{ref},k}) \right\|_Q^2 + \left\| u_k - u_{\mathrm{hover}}(p) \right\|_R^2 \right) \\ & \qquad + \left\| e_{\mathrm{track}}(x_N, x_{\mathrm{ref},N}) \right\|_P^2 \\ \text{s.t.} \quad & x_{k+1} = F(x_k, u_k; p_{\mathrm{NMPC}}), \\ & \underline{T} \leq T_k \leq \overline{T}, \quad |\tau_{k,i}| \leq \overline{\tau}_i, \end{aligned} \tag{4.19}$$

where F is one explicit fourth-order Runge–Kutta step of the prediction model (3.26) with the parameters (3.25), and the input bounds are those of Section 4.5.4. The problem is re-solved at every step; what is sent to the autopilot is not u_0 itself but the thrust and body-rate setpoints read from the predicted trajectory, as described in Section 3.5.3.

The second term penalises the deviation of the input from a *baseline* u_{hover}, recomputed from the live parameters at every solve rather than fixed when the solver is generated:

$$T_h = m_T\left(g - \hat{d}_z\right), \qquad u_{\text{hover}} = \left[T_h,\, 0,\, 0,\, 0\right]^\top. \tag{4.20}$$

With the additive residual of (3.26), vertical equilibrium requires $T/m_T - g + \hat{d}_z = 0$, which is (4.20); $\hat{d}_z$ is a specific force, so it is scaled by the mass. Penalising $\|u\|_R^2$ outright would treat the thrust that keeps the vehicle airborne as an expense; the baseline recentres the penalty on the equilibrium input, so that R charges only genuine control effort [1]. What is particular here is that the baseline is a function of the *estimated* mass. A stale baseline — for example one computed from the unloaded mass while a parcel is carried — pulls the thrust persistently below equilibrium, and the optimum settles at a non-zero steady-state error where the marginal tracking and baseline costs balance. The signature is that increasing R makes the error worse, because R is the strength of the incorrect pull; in development flights with a heavy load and an empty-airframe baseline the tracking error stalled above 0.85 m.

The torque entries of (4.20) are zero because a centred payload requires no trim moment. The small trim left by imperfect alignment is not represented in the controller's model or cost; it is supplied by the integral action of the autopilot's rate loop (Section 5.7.3).

4.5.2 Cost and weights

The controller solves (4.19) with $N = 20$ stages at $\Delta t = 0.05\,\text{s}$, preserving a 1.0 s preview while running at 20 Hz. The residual vector is

$$\begin{aligned} y_k = \big[& \boldsymbol{e}_p;\ \boldsymbol{e}_v;\ \boldsymbol{e}_{q,\text{vec}};\ \boldsymbol{e}_\omega; \\ & u_k - u_{\text{hover}}(p)\big] \in \mathbb{R}^{16}, \end{aligned} \tag{4.21}$$

where the attitude entry is the vector part of the relative quaternion $\boldsymbol{q}_{\text{ref}}^{*} \otimes \boldsymbol{q}$, which keeps the residual on the tangent space of the rotation manifold instead of treating four quaternion components as independent Euclidean coordinates.

Weights again follow Bryson's rule, $Q_{ii} = L_i^{-2}$, with tolerances

$$L_p = 0.1\,\text{m}, \quad L_v = 0.3\,\text{m}\,\text{s}^{-1}, \quad L_q = 0.05, \quad L_\omega = 0.3\,\text{rad}\,\text{s}^{-1}, \tag{4.22}$$

and, on the inputs, $L_T = 2\,\text{N}$, $L_{\tau,xy} = 0.5\,\text{N}\,\text{m}$, $L_{\tau,z} = 0.2\,\text{N}\,\text{m}$; the torque scales are taken to be the constraint bounds themselves, since the input cannot exceed them in any case. The terminal weight rescales the stage blocks by 20 (position), 1 (velocity), 2 (attitude) and 0.2 (rate): the endpoint of the prediction is pulled tightly onto the reference in position and attitude, while relaxing the terminal rate demonstrably helps the solver.

This non-dimensionalisation is not cosmetic. Before it was applied, position and torque weights differed by a factor of tens at their natural magnitudes, the Gauss–Newton Hessian was correspondingly ill-conditioned across directions, and the stationarity residual was observed to reach 6.26×10^5 — a complete numerical breakdown rather than a slow convergence.

Trajectory-scaled tolerances. Two of the entries in (4.22) are defined by the comment that accompanies them as trajectory-dependent quantities: L_v is a characteristic speed and L_ω a characteristic yaw rate. Written as constants they are correct only for the low-speed reference they were tuned on. An optional rescaling restores the intent,

$$L_v \leftarrow \max\big(L_v,\ r\,w\big), \qquad L_\omega \leftarrow \max\big(L_\omega,\ K_\psi\, w\big), \qquad K_\psi = 3.17, \tag{4.23}$$

taking the maximum so that the weights are only ever relaxed, never tightened. The constant K_ψ is the ratio of peak yaw rate to angular rate on the figure-eight of Section 4.7, obtained numerically; it is worth noting that evaluating

the yaw rate at the intuitive location $a = \pi/4$ gives 2.83 and underestimates the peak by 12 %, because the extremum does not occur there. Applying (4.23) at the low-speed condition reproduces L_v exactly but relaxes L_ω by a factor of 3.2, which exposes a pre-existing mismatch rather than introducing one: the original value was documented as matching a *circular* trajectory, whose yaw rate is constant at w, while the vehicle actually flies a figure-eight whose peak is $3.17w$. The rescaling is off by default so that historical batches remain reproducible.

4.5.3 Residual specific force

The translational model at the published mass is still wrong whenever the mass estimate is: under sustained manoeuvre the loaded estimate is biased low (Section 5.5.1), and the thrust feed-forward implied by (3.26) then falls short by the same proportion, leaving a steady-state height error that only the position error can push against. The controller therefore carries an additive specific force $\hat{\boldsymbol{d}}$ [$\mathrm{m\,s^{-2}}$], estimated at the odometry rate by the state predictor and adaptation law of $\mathscr{L}_1$-adaptive control [20]:

$$\dot{\hat{\boldsymbol{v}}} = \boldsymbol{a}_{\mathrm{nom}} + \hat{\boldsymbol{d}}_{\mathrm{raw}} - a\,(\hat{\boldsymbol{v}} - \boldsymbol{v}), \qquad \dot{\hat{\boldsymbol{d}}}_{\mathrm{raw}} = -\Gamma\,(\hat{\boldsymbol{v}} - \boldsymbol{v}), \qquad \hat{\boldsymbol{d}} = \frac{\omega_c}{s + \omega_c}\,\hat{\boldsymbol{d}}_{\mathrm{raw}}, \quad (4.24)$$

with $a = 10\,\mathrm{s^{-1}}$, $\Gamma = (a/2)^2$ (critically damped) and low-pass cut-off $\omega_c = 0.5\,\mathrm{rad\,s^{-1}}$. The nominal acceleration $\boldsymbol{a}_{\mathrm{nom}}$ is (3.7) evaluated with the *measured* collective thrust of (3.24) and the *published* mass $\hat{m}$, so $\hat{\boldsymbol{d}}$ absorbs only the part of the measured acceleration that the model at $\hat{m}$ fails to explain. It enters (3.26) additively and the baseline (4.20) through its vertical component.

Four properties keep it out of the payload accounting. It is a specific force, not a mass, so it needs no assumption about what the mass is. It is used by the controller only: it does not reach the estimator, the evidence score (4.13) or

any lifecycle decision. It is reset to zero when the controller accepts a release with a release command outstanding (Section 4.4.4), since a residual learned against the loaded vehicle becomes a phantom force once the payload has gone; an acceptance with *no* command outstanding does not reset it, and the residual is then unlearned by (4.24) itself, at the 2 s time constant of the low-pass filter. And it is held at zero while the measured thrust is below $0.4\,m_B g$, because on the ground the support force would otherwise be learned as an upward disturbance. Its cost is equally plain: because it compensates the translational consequence of a mass bias, tracking results obtained with it do not measure the accuracy of $\hat{m}$, and the mass error is reported directly wherever it matters (Chapter 5). For the same reason the $\mathscr{L}_1$ baseline, which uses the same law with a fixed nominal mass and no payload model, differs from the deployed controller in the payload model, not in the presence of disturbance estimation. Section 5.7.1 isolates its contribution.

4.5.4 Input constraints and the actuator envelope

The rotor model of Section 3.5 also defines what the vehicle can actually produce, and therefore the input constraint set of (4.19). The autopilot accepts a normalised throttle η, which the electronic speed controllers map affinely to rotor speed before the quadratic thrust law applies:

$$\omega(\eta) = \omega_{\min} + \Delta\omega\,\eta, \qquad T(\eta) = K\,\omega(\eta)^2, \qquad K = 4k_f, \tag{4.25}$$

with $\omega_{\min} = 150\,\mathrm{rad\,s^{-1}}$ and $\Delta\omega = 850\,\mathrm{rad\,s^{-1}}$. Inverting (4.25) gives the command actually published,

$$\eta(T) = \frac{\sqrt{T/K} - \omega_{\min}}{\Delta\omega}, \tag{4.26}$$

which is exact at every mass, unlike a linear hover-referenced heuristic. Because η is clipped to $[0.05, 0.95]$ — the upper limit reserving 5 % of the command

range so that the rate loop retains differential authority when all four rotors are near saturation — the attainable collective thrust is

$$T \in \big[\,T(0.05),\, T(0.95)\,\big] = [\,1.27\,\mathrm{N},\, 31.35\,\mathrm{N}\,], \tag{4.27}$$

and this is the thrust box of (4.19). A box derived from the vehicle's weight rather than its actuators, as used in an earlier implementation ($[0.5\,\mathrm{N}, 2m_{BG} = 40.5\,\mathrm{N}]$), over-states the available thrust by 29 %; the surplus is removed silently by the clip in (4.26), so a planned recovery acceleration is never delivered and the next solve asks for still more.

The torque bounds follow from the allocation. The ROS layer never evaluates the inverse of (3.24) — the autopilot's mixer does — but its form sizes the bounds. The rows of M are mutually orthogonal, so $M^{-1} = M^{\top}\mathrm{diag}\big(\frac{1}{4}, \frac{1}{4l^2}, \frac{1}{4l^2}, \frac{1}{4k_m^2}\big)$ and

$$F_i = \frac{T}{4} + \frac{y_i}{4l^2}\,\tau_x - \frac{x_i}{4l^2}\,\tau_y - \frac{d_i}{4k_m}\,\tau_z. \tag{4.28}$$

A unit roll or pitch moment costs $1/(4l) = 1.44\,\mathrm{m}^{-1}$ of differential rotor thrust, whereas a unit yaw moment costs $1/(4k_m) = 15.6\,\mathrm{m}^{-1}$, a factor of 10.9. At a hover thrust of 20 N each rotor carries 5 N and has 2.84 N of headroom to the 7.84 N per-rotor ceiling implied by (4.27); that headroom buys 1.98 N m of roll authority but only 0.18 N m of yaw. Against the bounds $\overline{\tau}_{x,y} = 0.5\,\mathrm{N\,m}$ and $\overline{\tau}_z = 0.2\,\mathrm{N\,m}$ used in (4.19), the roll and pitch bounds are conservative by a factor of four while the yaw bound is marginally *optimistic*, which is consistent with the executed yaw torque falling well short of the intended one (Section 5.1).

4.5.5 Solver

Both optimal control problems are transcribed by direct multiple shooting with explicit fourth-order Runge–Kutta integration, a Gauss–Newton Hessian

approximation, and partial-condensing HPIPM [10] as the quadratic subproblem solver, generated and solved with acados [8]. Both use *full* sequential quadratic programming with merit-function backtracking rather than the real-time-iteration scheme [6]: RTI was observed to diverge silently in this configuration, and a controller that reports success while producing a diverging input is worse than one that takes an extra millisecond. Generated C code is cached and reused until the model or problem dimensions change.

4.6 Inner-Loop Scheduling

Model consistency in the outer loop cannot restore inner-loop bandwidth. The autopilot's rate controller is tuned for the bare airframe; a payload that multiplies the roll and pitch inertia by κ of (3.15) lowers its effective bandwidth by the same factor, and for heavy loads the resulting attitude oscillation grows until the torque saturates. At the nominal arm κ is 2.34 for the heavier payload, by (3.16); because it grows with r_z^2, the arm and the payload mass together define a feasibility boundary, characterised in Section 5.8.3. An earlier implementation applied κ by rewriting the autopilot's rate-loop gains at attachment. The deployed implementation does not touch those parameters; it scales the body-rate *command* instead, which keeps the adaptation inside the ROS layer and makes it continuous and reversible.

4.6.1 From inertia ratio to a command scale

The deployed response scales the body-rate *command* around the current measured rate,

$$\omega'_{\mathrm{cmd}} = \omega + \varsigma\left(\omega_{\mathrm{cmd}} - \omega\right), \qquad \varsigma \geq 1, \tag{4.29}$$

applied to the roll and pitch channels only, since ΔJ enters only J_{xx} and J_{yy}. Equation (4.29) exaggerates the rate *error* the inner loop sees, which is equivalent to raising its proportional gain, without modifying any autopilot parameter. Because it is a common scale on the roll/pitch pair it preserves the direction of the rate-error vector.

The requested scale is the inertia ratio (3.15), clipped at a cap of 5.0, evaluated from the live ΔJ of the payload frame. Three filters stand between the request and its application, each addressing a distinct failure:

- a **dead band** of 0.08 on the target, so that estimator noise cannot move it, with explicit snapping to 1.0 and to the cap so that hysteresis cannot strand the empty-airframe steady state permanently above unity;
- a **first-order filter** with $\tau = 0.5\,\mathrm{s}$, chosen far slower than the inner loop (hundreds of hertz) and still fast enough to follow the second-scale progressive load transfer during a grasp;
- **asymmetric rate limits**, $4.0\,\mathrm{s}^{-1}$ rising against $1.5\,\mathrm{s}^{-1}$ recovering, encoding the asymmetry of the consequences: being late to add authority risks saturation, being late to remove it does not.

Because ΔJ is a function of $\hat{m}$, the scale is too: the inner loop receives exactly the authority the mass estimate implies, and a mass estimate that collapses toward the bare airframe during manoeuvre withdraws that authority at the moment it is needed (Section 4.4.6).

4.6.2 Headroom limiting

Scaling a command that is already close to the rate limit accomplishes nothing except saturation, at which point the scaling has silently stopped acting. The

implementation therefore solves for the largest scale that keeps the scaled command inside a guard band $\pm L$, with L equal to 95 % of the hard rate limit $\overline{\omega} = 2.0\,\mathrm{rad\,s^{-1}}$.

For one channel, write the rate error $e = \omega_{\mathrm{cmd}} - \omega$. Requiring $|\omega + \varsigma e| \le L$ gives

$$\varsigma \;\le\; \begin{cases} \dfrac{L-\omega}{e}, & e > 0, \\[2ex] \dfrac{-L-\omega}{e}, & e < 0, \end{cases} \tag{4.30}$$

and the applied scale is the smallest such bound over the roll and pitch channels, floored at unity:

$$\varsigma_{\mathrm{eff}} = \max\Big(1,\; \min_{i\in\{x,y\}} \varsigma_i^{\max}\Big). \tag{4.31}$$

The floor at unity guarantees that the scheduler can never make the raw controller command *less* authoritative than it would have been without scheduling — the scheduler is allowed to help and not allowed to hurt. When (4.31) binds, it is logged, so that saturation avoidance is an observable event rather than a silent degradation.

4.6.3 Attach-time inertia bootstrap

One interval resists estimate-driven adaptation: the second or so between a load beginning to take weight and the estimator accumulating enough evidence to say so. During that interval the vehicle is physically loaded and the model is not, and it is the interval in which the predecessor implementation lost eleven of sixteen flights at the heaviest condition.

The deployed bootstrap addresses it with the narrowest intervention that works. When the grasp is enabled, the *inertia* parameter alone is floored at the value implied by the platform's 0.30 kg payload envelope; the mass is untouched, and no payload is fabricated in it. At the nominal arm the floor is $\Delta J =$

$0.0191\,\mathrm{kg\,m^2}$, an inner-loop ratio of 2.34; the controller uses the larger of the floor and the estimate-derived increment, so a heavier estimate still raises the inertia. Two hand-over rules are implemented. In the original rule the overlay is faded out over 0.5 s once the estimator independently reports persistent loaded evidence, namely $\Delta J \geq 0.010\,\mathrm{kg\,m^2}$ together with an evidence score well below the confirmation threshold, sustained for 0.5 s. In the rule used for evaluation (*hold until command*), loaded evidence does not end the floor; it is held until the controller issues the release command and is then faded over the same 0.5 s to the estimate-derived increment. If the estimate becomes stale or unhealthy while the handover is in progress, the fade freezes at the last safe inertia rather than continuing toward zero.

The hold rule is the one place where the release command touches the controller's payload model, and it should be stated exactly. The command ends an *overlay*; it does not assert that the payload has gone. After the fade the inertia is whatever the estimate implies, which for a payload still attached is the loaded value, and the release is still accepted only by (4.14). The command is waited for because, after a real release, the estimator can no longer produce the loaded evidence that would end the floor, and the floor would otherwise remain on an empty airframe indefinitely. By the same token, a load lost without any command leaves the floor on the empty airframe; that case is outside the evaluation (Section 5.3.2).

Three properties keep this consistent with the rest of the design. It is *inertia-only*, so it cannot influence the mass channel on which every release decision depends. It can only *raise* the inertia above what the estimate implies, never lower it, so the release path remains estimate-only. And its magnitude is the airframe's payload envelope, which Section 3.4.4 has already disclosed as a platform specification rather than task information.

The bootstrap also has a known failure of its own. It is armed when the grasp is enabled, whether or not the gripper actually catches the parcel, and

it is handed over only on loaded evidence. A missed grasp therefore leaves an empty airframe flying with the envelope inertia floor — a 2.34-fold inner-loop scale at the nominal arm — because the evidence that would release the floor never arrives; under the hold rule it is in any case kept until the release command. A variant that arms the floor only once the vehicle has detectably lifted the load is implemented and off by default.

4.7 Reference Trajectory

The evaluation trajectory is a three-dimensional Gerono lemniscate,

$$x_r = r\sin a, \qquad y_r = r\sin a\cos a = \tfrac{r}{2}\sin 2a, \qquad z_r = z_h + d_z\sin a, \qquad a = wt_c, \tag{4.32}$$

with yaw commanded along the velocity vector. The crossing is not a cusp: $\boldsymbol{v} = rw(\cos a, \cos 2a)$ has magnitude $\sqrt{2}rw$ there while the planar acceleration vanishes. The curve therefore excites coupled translation and yaw without a velocity reversal, and the vertical term keeps the workspace compact.

After the initial hover, position amplitude and heading are introduced with the quintic smooth step $\alpha(s) = 10s^3 - 15s^4 + 6s^5$. The ramp duration satisfies $t_{\text{ramp}} \geq 2.65/w$, limiting the extra velocity due to amplitude growth, and heading is blended from the hover direction to the trajectory tangent. This yaw blend was retained because, without it, six development flights at $4\,\mathrm{m\,s^{-1}}$ spent 19–25 % of their first 5 s at the yaw-moment constraint.

4.8 Online Execution Sequence

At every control instant the implementation performs the following sequence.

1. Receive and frame-convert the latest odometry.
2. Reconstruct the physical wrench from rotor speeds, (3.24), averaging the 250 Hz samples over the estimation interval.
3. Append the measurement and the reconstructed input to the 10 Hz window; re-seed by (4.9) if the window must be rebuilt.
4. Solve (4.1) for m_T and the physical trajectory.
5. Update the presence machine and the no-payload evidence score (4.13).
6. Assemble and publish the atomic frame (4.10) with its health bit and solution age.
7. In the controller: validate freshness, low-pass filter and slew-limit the frame (4.11), and write m_T and ΔJ into the model parameters and the hover baseline (4.20).
8. Evaluate confirmation (4.14) and the UNRESOLVED timeout; switch the reference to the braking quintic (4.16) if the timeout expires.
9. Shift the previous solution forward as a warm start and solve (4.19). Between solves, publish at 50 Hz the thrust and the body rate interpolated along the predicted trajectory (Section 3.5.3); invert thrust to throttle by (4.26) and apply the body-rate scaling (4.29)–(4.31).
10. Log estimates, residuals, constraint activity, solver statistics and every lifecycle transition.

Estimator and controller failures are handled separately and asymmetrically. An invalid estimator solve retains the last accepted parameter set and marks the frame unhealthy, which — by Section 4.3.1 — freezes the controller's model and resets its confirmation timer. An invalid controller solve falls back to the configured safe action rather than publishing an unconstrained command.

4.9 Chapter Summary

This chapter addressed the third and fourth research objectives: turning the single-parameter model of Chapter 3 into an estimator and a controller that run together, and defining the release lifecycle. The estimator is a constrained moving-horizon problem whose only payload decision variable is the composite mass, initialised from a window force balance and driven by the measured wrench. One atomic, health-gated frame carries the mass and the inertia increment derived from it to the controller, which updates its prediction model, its input baseline and its inner-loop command scaling continuously from that frame. Release acceptance is kept separate from that adaptation: it changes the lifecycle state, resets the residual force and declares completion, and it is gated on estimator evidence. Two properties of the design are known before any experiment: the lifecycle rests on the mass channel alone, and the attach-time inertia floor protects the inner loop at the price of not being released by evidence. Chapter 5 measures how the complete system behaves, including whether the always-on or the command-armed confirmation policy should be used.